

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

Seaport

0x000000000000000068F116a894984e2DB1123eB395

Ethereum

Compiler: v0.8.24+commit.e11b9ed9

Date: October 7, 2026

OVERALL RISK RATING *AI-assisted audit. Not a substitute for manual review.*

Low

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. This is the official Seaport 1.6 marketplace contract set, a mature, heavily audited, and formally verified codebase with no owner-gated functions and no upgradeability. The implementation relies extensively on hand-optimized inline assembly and custom calldata/memory pointer libraries to minimize gas costs, which increases code complexity and the theoretical risk of subtle memory-safety bugs, but no exploitable critical/high severity issues were identified in the reviewed files. Findings below are limited to low-severity and informational items related to design tradeoffs, floating pragma, and external call patterns that are largely intentional and already mitigated by the protocol's architecture (reentrancy guards, EIP-712 domain separation, zone/offerer opt-in trust model).

Findings (6)

F-001 · SWC-103

Low

Floating pragma used across all contracts

src/Seaport.sol:2–2

DESCRIPTION

All reviewed contracts declare ``pragma solidity ^0.8.24;`` (or ``^0.8.13`/^0.8.14`/^0.8.17`` in shared libraries), allowing compilation with any compiler version satisfying the caret range. Deploying with an untested/newer compiler version could introduce unexpected codegen changes, particularly given the codebase's heavy reliance on inline assembly and manual memory layout assumptions that depend on specific compiler behavior (e.g., free memory pointer handling).

IMPACT

Low, a future compiler version could alter optimizer behavior or ABI encoding in ways that are incompatible with the manually managed memory layouts used throughout the Consideration/Encoder/Decoder libraries, though this is mitigated by the fact production deployment is pinned to a specific compiler (v0.8.24).

RECOMMENDATION

Lock the pragma to the exact compiler version used for the audited/deployed bytecode (e.g. ``pragma solidity 0.8.24;``) in all first-party files to guarantee reproducible builds.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import { Consideration } from "../lib/Consideration.sol";
```

F-002 · SWC-134

Low

Unbounded external calls to zones/contract offerers within fulfillment loops

src/lib/OrderCombiner.sol:330–410

DESCRIPTION

When fulfilling restricted or contract orders via ``fulfillAvailableAdvancedOrders`/^matchAdvancedOrders``, Seaport performs external calls (``authorizeOrder``, ``validateOrder``, ``ratifyOrder``, ``generateOrder``) to zone/off-ferer-controlled addresses inside loops over the orders array, with no explicit gas stipend limit beyond the

overall transaction gas budget. A malicious or buggy zone/contract-offerer included in a batch could consume excessive gas or revert, aborting the whole batch for that caller.

IMPACT

Impact is restricted to the fulfiller's own transaction (self-inflicted griefing) since only orders the caller voluntarily selects to fulfill are affected; it is not an attack vector against other users' funds or state, but it is a known operational risk when aggregating many restricted/contract orders together.

RECOMMENDATION

This is an accepted design tradeoff intrinsic to supporting restricted and contract order types; no code change is required. Integrators building aggregation UX on top of Seaport should pre-validate zones/contract offerers (e.g., via simulation) before batching untrusted orders together, and could optionally supply explicit gas limits off-chain when constructing calls to fulfill functions with many restricted orders.

```
if (containsNonOpen) {
  for (uint256 i = 0; i < totalOrders; ) {
    if (availableOrders[i]) {
      _assertRestrictedAdvancedOrderValidity(
        advancedOrders[i], orderHashes, orderHashes[i]
      );
    }
    unchecked { ++i; }
  }
}
```

F-003

Informational

Publicly callable TSTORE activation function with no access control

src/lib/ReentrancyGuard.sol:77-101

DESCRIPTION

`\_\_activateTstore()` is an unauthenticated external function that any address may call once to switch the reentrancy guard storage mechanism from SSTORE to TSTORE. While the function already contains internal guards preventing re-activation or activation while reentrancy is set, allowing any caller to trigger this state transition is an intentional but unusual design choice worth flagging for completeness.

IMPACT

No direct fund-loss impact; the function cannot be abused to bypass the reentrancy guard and simply optimizes gas usage for all future calls once activated. Its asymmetric lack of access control is a deliberate design decision (anyone benefits from lower gas once TSTORE is confirmed supported).

RECOMMENDATION

No change required; document explicitly (as already done in NatSpec) that this is a one-time, permissionless, idempotent-by-design activation function with no security implications.

```
function __activateTstore() external {
  bool tstoreActivatable;
  assembly {
    tstoreActivatable := eq(sload(_REENTRANCY_GUARD_SLOT), _NOT_ENTERED_SSTORE)
  }
  if (_tstoreInitialSupport || !tstoreActivatable) {
    revert TStoreAlreadyActivated();
  }
  ...
}
```

Quasi-random counter increment uses blockhash as entropy source

src/lib/CounterManager.sol:34–40

DESCRIPTION

``_incrementCounter`` derives a pseudo-random increment from ``blockhash(block.number - 1)``. Block hashes are influenceable/predictable by miners/validators to a limited degree.

IMPACT

No security impact: the randomness is used only to make the resulting counter value unpredictable/non-sequential so that previously signed orders cannot be trivially "reactivated"; it is not used for any economically exploitable selection (e.g., lottery, auction winner) and an attacker gains nothing from predicting or influencing it, since the offerer alone can call this function for their own counter.

RECOMMENDATION

No action required; this is a benign, well-documented use of blockhash and is not a weak-randomness vulnerability in the traditional sense.

```
let quasiRandomNumber := shr(
  Counter_blockhash_shift,
  blockhash(sub(number(), 1))
)
...
newCounter := add(quasiRandomNumber, sload(storagePointer))
```

Extensive use of low-level inline assembly and manual memory management

src/lib/OrderCombiner.sol

DESCRIPTION

The majority of the fulfillment, aggregation, and encoding/decoding logic (OrderCombiner, FulfillmentApplier, ConsiderationEncoder/Decoder, TokenTransferrer, GettersAndDerivers) is implemented in raw Yul/inline assembly that directly manipulates memory pointers, the free-memory pointer, and calldata offsets, including manual repurposing of struct fields (e.g., OfferItem fields overloaded to behave as ReceivedItem fields) to save gas.

IMPACT

This dramatically increases the cognitive overhead required to verify correctness and makes the code more fragile to future modifications (e.g., changing struct layouts, Solidity ABI encoding assumptions, or compiler memory-safety guarantees could silently break invariants). Given this is the official, extensively tested and audited Seaport source, no functional defects were identified, but the pattern itself carries elevated maintenance risk for any forks or modifications.

RECOMMENDATION

For any customized fork or modification of this code, commission a full assembly-level audit and extensive differential fuzz testing against the reference implementation before deployment, since even minor edits to memory offsets/struct layouts can introduce critical corruption bugs that are not caught by standard Solidity-level review.

```
assembly {
  mstore(
    add(considerationItem, ReceivedItem_recipient_offset),
    mload(considerationItemRecipientPtr)
  )
}
```

```
mstore(considerationItemRecipientPtr, currentAmount)
}
```

F-006

Informational

Trust dependency on external ConduitController and conduit implementations

src/lib/GettersAndDrivers.sol:163–196

DESCRIPTION

`\_deriveConduit` computes a conduit address via CREATE2 using an immutable creation-code hash fetched from the supplied `conduitController` at construction time. Conduits are externally owned/controlled contracts (not deployed/managed by Seaport itself) that, once granted an open channel, can transfer any token for which a user has granted approval to that conduit.

IMPACT

If a user approves a malicious or compromised conduit, or if a conduit owner opens a channel to a malicious caller, funds approved to that conduit can be drained. This is a known, documented trust boundary of the Seaport/conduit architecture and not a defect in the Seaport contract itself, it is an explicit design choice to allow flexible approval delegation.

RECOMMENDATION

No code change needed in Seaport; this finding is purely informational for integrators and end users: only approve well-audited conduits (e.g., the canonical OpenSea conduit) and verify channel ownership/configuration before granting token approvals.

```
conduit := and(
  keccak256(Create2AddressDerivation_ptr, Create2AddressDerivation_length),
  MaskOverLastTwentyBytes
)
```

Gas Optimizations

Codebase is already heavily gas-optimized via assembly

`src/lib/OrderFulfiller.sol`

No further gas optimizations are recommended; the contract set already uses custom calldata/memory pointer libraries, accumulator-based conduit batching, and struct field repurposing specifically to minimize gas costs. Any changes should be benchmarked against the existing reference gas snapshots before being adopted.

Appendix - Metadata

Contract	Seaport
Address	0x000000000000000068F116a894984e2DB1123eB395
Chain	Ethereum
Compiler	v0.8.24+commit.e11b9ed9
Audit ID	portfolio-seaport-1-6
Date	10/7/2026

Files analyzed

```
[user] src/Seaport.sol
[user] src/lib/Consideration.sol
[user] lib/seaport-types/src/interfaces/ConsiderationInterface.sol
[user] lib/seaport-types/src/lib/ConsiderationStructs.sol
[user] src/lib/OrderCombiner.sol
[user] lib/seaport-types/src/helpers/PointerLibraries.sol
[user] lib/seaport-types/src/lib/ConsiderationConstants.sol
[user] lib/seaport-types/src/lib/ConsiderationEnums.sol
[user] src/lib/OrderFulfiller.sol
[user] src/lib/FulfillmentApplier.sol
[user] lib/seaport-types/src/lib/ConsiderationErrors.sol
[user] lib/seaport-types/src/lib/ConsiderationErrorConstants.sol
[user] src/lib/BasicOrderFulfiller.sol
[user] src/lib/CriteriaResolution.sol
[user] src/lib/AmountDeriver.sol
[user] lib/seaport-types/src/interfaces/FulfillmentApplicationErrors.sol
[user] src/lib/OrderValidator.sol
[user] lib/seaport-types/src/interfaces/CriteriaResolutionErrors.sol
[user] lib/seaport-types/src/interfaces/AmountDerivationErrors.sol
[user] src/lib/Executor.sol
[user] src/lib/ZoneInteraction.sol
[user] lib/seaport-types/src/interfaces/ConduitInterface.sol
[user] lib/seaport-types/src/conduit/lib/ConduitEnums.sol
[user] src/lib/Verifiers.sol
[user] src/lib/TokenTransferrer.sol
[user] lib/seaport-types/src/interfaces/ZoneInteractionErrors.sol
[user] src/lib/LowLevelHelpers.sol
[user] src/lib/ConsiderationEncoder.sol
[user] lib/seaport-types/src/conduit/lib/ConduitStructs.sol
[user] src/lib/Assertions.sol
[user] src/lib/SignatureVerification.sol
[user] lib/seaport-types/src/lib/TokenTransferrerConstants.sol
[user] lib/seaport-types/src/interfaces/TokenTransferrerErrors.sol
[user] src/lib/GettersAndDerivers.sol
[user] src/lib/CounterManager.sol
[user] lib/seaport-types/src/interfaces/SignatureVerificationErrors.sol
[user] src/lib/ConsiderationBase.sol
```

```
[user] lib/seaport-types/src/interfaces/ConsiderationEventsAndErrors.sol
[user] src/lib/ReentrancyGuard.sol
[user] lib/seaport-types/src/interfaces/ConduitControllerInterface.sol
[user] src/lib/ConsiderationDecoder.sol
[user] lib/seaport-types/src/interfaces/ReentrancyErrors.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-seaport-1-6)

<https://cryptohawking.com/audit/manual?ref=portfolio-seaport-1-6>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>